

Contenu à coller dans l'Annexe VII-1-B (SLAM)

RECTO

Intitulé de la réalisation professionnelle : Application web de gestion d'inventaire du parc informatique

Modalité : Seul(e)

Compétences travaillées : (cocher les 3) - ☒ Concevoir et développer une solution applicative - ☒ Assurer la maintenance corrective ou évolutive d'une solution applicative - ☒ Gérer les données

Conditions de réalisation (ressources fournies, résultats attendus) :

L'équipe IT de l'organisation avait besoin d'un outil interne pour centraliser la gestion de son parc informatique (PCs, laptops, serveurs). Aucun outil existant n'était en place. Ressources fournies : cahier des charges fonctionnel, accès à un environnement serveur avec Docker. Résultats attendus : une application web sécurisée permettant de lister, ajouter, modifier et supprimer des postes de travail, d'importer des inventaires en masse via CSV, d'afficher des statistiques et d'administrer les comptes utilisateurs avec un système de permissions granulaire.

Description des ressources documentaires, matérielles et logicielles utilisées :

- Backend : PHP 8.0 natif (sans framework)
- Base de données : MariaDB, accès via PDO (prepared statements)
- Frontend : Bootstrap 5.3, Bootstrap Icons, JavaScript ES6 vanilla
- Bibliothèque JS : SortableJS 1.15.6 (drag & drop)
- Conteneurisation : Docker Compose (conteneur PHP/Apache + conteneur MariaDB)
- Gestion de versions : Git
- Éditeur : Visual Studio Code

- Navigateurs de test : Chrome, Firefox

Modalités d'accès aux productions et à leur documentation :

Application accessible via `http://localhost:8080` après lancement avec `docker compose up`. Compte par défaut : admin / root. Code source accessible via Git. Documentation dans le dossier `/docs` : cahier des charges, fiche technique, explication du code. Schéma de base de données dans le dossier `/database`.

VERSO

Contexte et besoin

L'organisation IT ne disposait d'aucun outil centralisé pour suivre son parc informatique. Les informations étaient dispersées dans des fichiers tableur non partagés. L'objectif était de développer une application web interne sécurisée, accessible à plusieurs techniciens avec des niveaux de permissions différents.

Architecture technique

Application PHP 8.0 native sans framework, déployée via Docker Compose sur deux conteneurs : PHP/Apache et MariaDB. Interface Bootstrap 5.3 responsive. Schéma SQL modulaire en 4 fichiers : table `pcs` (inventaire), table `users` (comptes), tables `custom_fields` et `pc_custom_data` (champs personnalisés), table `field_options` (listes déroulantes).

Gestion de l'inventaire (CRUD)

Liste paginée (50 entrées/page) avec filtres multi-critères : recherche textuelle sur 8 colonnes, filtre par statut, architecture et marque. Tri cliquable sur toutes les colonnes. Ajout et modification via formulaire avec cascades JavaScript (marque → modèle, OS → version OS). Suppression unitaire et suppression en lot multi-pages avec sélection persistée en `localStorage`.

Tableau de bord

Compteurs par statut, répartition par architecture (barres de progression), top 5 marques et OS, 10 dernières modifications, 5 derniers ajouts. 6 requêtes SQL distinctes au chargement.

Import CSV

Traitement ligne par ligne avec UPSERT (INSERT ... ON DUPLICATE KEY UPDATE sur le numéro de série). Génération automatique du numéro de série si absent. Validation des colonnes obligatoires et des valeurs d'énumération. Synchronisation automatique des nouvelles valeurs dans les listes déroulantes après import.

Scripts de collecte automatique

Script PowerShell (Windows) et script Bash (Linux) téléchargeables depuis l'interface admin. Ils collectent automatiquement les informations du poste local et génèrent un fichier inventaire.csv importable directement dans l'application.

Administration

Gestion des champs : ajout de champs personnalisés, masquage, réordonnancement par drag & drop (SortableJS + AJAX). Gestion des 4 listes déroulantes (marques, modèles, OS, versions OS). Gestion des comptes avec 5 permissions indépendantes (can_view, can_add, can_edit, can_delete, is_admin). Statistiques PCs par utilisateur avec barres de progression et recherche live.

Sécurité

Authentification par session PHP avec session_regenerate_id() à la connexion, mots de passe bcrypt. Protection CSRF sur tous les formulaires POST (token random_bytes vérifié via hash_equals). Prévention XSS : toutes les sorties HTML passent par htmlspecialchars. 100% PDO paramétré, whitelist des colonnes de tri, validation stricte des énumérations. Vérification des permissions côté serveur sur chaque page.

Gestion des données

Clé UNIQUE sur le numéro de série (fondement de l'UPSERT). Contrainte ON DELETE CASCADE sur les données personnalisées. Index sur hostname et utilisateur. Pagination côté serveur (COUNT + LIMIT/OFFSET). Requêtes préparées réutilisées en boucle pour les imports en masse.